

DS4MAZE

LAB 4
SECTION H

SUBMITTED BY:

Simon Samuel

SUBMISSION DATE:

11/21/2025

Problem

The aim of this lab was to create a basic game that utilizes a DualShock 4 controller to navigate an avatar through a randomly generated maze. In phase 1 we were to create the maze and make the avatar start from the top centre and slowly fall down. In Part 2, we added the ability for a controller to move the character via tilt. The game will end when the player reaches the bottom, and then you are presented with a YOU WIN! output. The game will also terminate if the player's avatar gets stuck on the walls and then the player loses the game. Ncurses was used to draw everything on the terminal screen.

Analysis

This program receives maze difficulty input from the command line (Initial for Mac users during the launch). Using scanf we read 8 parameters from the DualShock 4 – time, gx, gy, gz and the four button values. The output was the game running in real time on the screen. Using 2D array, maze is stored in the form of * and empty space will represent empty cells. The gx value from the controller was used to detect tilt. If gx was high/positive, the avatar tried to move left. If gx was low/negative, the avatar moved right. The time value helped control how fast the avatar fell. The controller inputs had an immediate effect on how the avatar moved around in the maze (direct relationship).

Design

To finish this lab, I had to work on part one first. For part one I made a function called generate_maze() that created the maze using random numbers. Then I used draw_maze() to display it on the screen with ncurses. The avatar was drawn using a provided function called draw_character(). In Part 2, I added a loop that kept reading values from input. The program constantly checked if it was safe to move left, right, or down before updating the avatar's position. I used a tolerance value so that tiny movements didn't trigger a direction change. To prevent crashing into walls, I checked the next position in the array before moving. Falling was based on a delay that used the DS4 time value. If the avatar made it to the bottom row, the game printed "YOU WIN!" and ended.

Testing

To test, I used multiple difficulties when I ran the code. The lower the difficulty the less * were printed, making it easier to solve also, decreasing the likely hood of the character getting stuck. When the difficulty was high, there were many more places for the character to get stuck. Also, for example at difficulty 100 the character spawned in stuck making it so you immediately lost. Testing also helped to find a proper tolerance and delay value that would make the game more enjoyable, because both helped to dictate the speed and sensitivity of the character's movement. So if it was too high or low, it would make the movement either too fast, or even too slow.

Comments

This lab helped me understand how sensors and input devices can control programs in real time. One issue I faced while working on the program was choosing a delay and a tolerance that would work well with each other. If the tolerance was too small the character would move too much and at the slightest movement. If the delay was too small, the character would move too quickly, and it was very difficult to even play the game. It took a bit of trial and error to find the correct balance.

Questions and Experiments

1. To verify these conditions the code checked the gx value for the controllers tilt. It then looked to see if that value was greater than or less than the tolerance to dictate which direction it was being tilted. Then before moving the character in which ever direction the controller was tilted in, the program checked the maze array to make sure there wasn't a cell wall, which prevented the character from moving into the walls. For the charter to fall, the code checked the time value to the last fall time value, and if enough time past, the avatar would fall.
2. If the avatar couldn't move, down, right, and left all at the same time, the program, knows that you have lost. This means the character was stuck, and the program ends uncurses and prints a message saying that the person has lost. In the state machine, this would come after the checking of movement, leading to a final 'Lose' state when no movement is available

Implementation

```
#include <stdio.h>

#include <stdlib.h>

#include <math.h>

#include <ncurses/ncurses.h>

#include <unistd.h>


#define NUMCOLS 100

#define NUMROWS 72


#define AVATAR 'A'

#define WALL '*'

#define EMPTY_SPACE ' '

```

```
char MAZE[NUMROWS][NUMCOLS];

// Prototypes
void generate_maze(int difficulty);
void draw_maze(void);
void draw_character(int x, int y, char use);

int main(int argc, char* argv[])
{
    if (argc < 2) {
        printf("Usage: ./lab8 difficulty\n");
        return 1;
    }

    int difficulty = atoi(argv[1]);

    // Ncurses setup
    initscr();
    noecho();
    curs_set(0);
    refresh();

    // Maze generation
    generate_maze(difficulty);
```

```

draw_maze();

// Avatar starting position
int x = COLS / 2;
int y = 0;
draw_character(x, y, AVATAR);

// DS4 variables
double t, gx, gy, gz;
int T_btn, C_btn, X_btn, S_btn;

// Timing for fall
double prevFallTime = 0;
int FALL_DELAY = 300;

// Tilt sensitivity
double tolerance = 0.5;

// GAME LOOP

while (1)
{
    // Read 8 values from ds4rd
    if (scanf("%lf,%lf,%lf,%lf,%d,%d,%d,%d",
        &t, &gx, &gy, &gz,

```

```

    &T_btn, &C_btn, &X_btn, &S_btn) != 8)

{
    continue;
}


// LEFT MOVEMENT
if (gx > tolerance)
{
    if (x > 0 && MAZE[y][x - 1] == EMPTY_SPACE)
    {
        draw_character(x, y, ' ');

        x--;

        draw_character(x, y, AVATAR);
    }
}


// RIGHT MOVEMENT
if (gx < -tolerance)
{
    if (x < COLS - 1 && MAZE[y][x + 1] == EMPTY_SPACE)
    {
        draw_character(x, y, ' ');

        x++;

        draw_character(x, y, AVATAR);
    }
}

```

```
}  
}
```

```
// FALLING  
if ((t - prevFallTime) > FALL_DELAY)  
{  
    if (y < LINES - 1 && MAZE[y + 1][x] == EMPTY_SPACE)  
    {  
        draw_character(x, y, ' ');  
        y++;  
        draw_character(x, y, AVATAR);  
    }  
    prevFallTime = t;  
}
```

```
// WIN CONDITION  
if (y >= LINES - 1)  
{  
    endwin();  
    printf("YOU WIN!\n");  
    return 0;  
}
```

```

// BONUS 1

int canMoveLeft = (x > 0      && MAZE[y][x - 1] == EMPTY_SPACE);

int canMoveRight = (x < COLS - 1  && MAZE[y][x + 1] == EMPTY_SPACE);

int canMoveDown = (y < LINES - 1 && MAZE[y + 1][x] == EMPTY_SPACE);


if (!canMoveLeft && !canMoveRight && !canMoveDown)
{
    endwin();

    printf("YOU LOSE - You got stuck!\n");

    return 0;
}

}

endwin();

return 0;

}


// Maze creation

void generate_maze(int difficulty)
{
    for (int r = 0; r < NUMROWS; r++)
    {
        for (int c = 0; c < NUMCOLS; c++)
        {

```

```
        if (rand() % 100 < difficulty)
            MAZE[r][c] = WALL;
        else
            MAZE[r][c] = EMPTY_SPACE;
    }
}
}
```

```
// Draw maze
```

```
void draw_maze(void)
{
    for (int r = 0; r < LINES; r++)
        for (int c = 0; c < COLS; c++)
            mvaddch(r, c, MAZE[r][c]);

    refresh();
}
```

```
// Provided: DO NOT MODIFY
```

```
void draw_character(int x, int y, char use)
{
    mvaddch(y, x, use);
}
```

```
refresh();
```

```
}
```